Evolving Slot Machine Reel Strips Under Noisy Simulation: When Does a Genetic Algorithm Beat Local Search?

Anonymous Author(s)

Anonymous Institution

Abstract. A video slot's main core structure, its reel strips, controls how much a game pays and how the game feels. Slot mathematicians tune these reel strips by hand and run millions of simulated spins after each change. We ask whether a genetic algorithm (GA) can outperform local search. For this, on one slot game, we have compared 9 configurations of GA with hill climbing, simulated annealing and random search, all of them with an equal number of evaluations and 4 mutation strengths, run over 30 paired seeds. In the base stage, no GA configuration could significantly beat local search, with the best GA having a median error of 0.076 against simulated annealing's 0.075. Generational replacement is significantly worse, losing to steady-state replacement on 30 of 30 seeds at the highest mutation strength ($A_{12} = 1.00$). In the free stage, no GA beats simulated annealing, and the targets appear to conflict.

Keywords: genetic algorithms · noisy optimisation · simulation-based optimisation · game design · slot machines

1 Introduction

A video slot is defined and controlled by its reel set, a set of strips of symbols. Hence mathematicians have to tune each reel set by hand to control the return to player (RTP) of the game.

There are three main difficulties in tuning. First, a large search space of 9^{250} possible reel sets. Second is noisy simulation. Each evaluation of 10^7 spins gives an error with a standard deviation of around 0.013 for a good base reel set, which is close to the gap between good candidates. Third, there are several targets, and changing one symbol can alter all of them.

A genetic algorithm (GA) [4] and local search [13] both were used to tune reel sets. Since a GA needs more design choices, such as a population, a selection method, a replacement scheme and crossover, we have asked whether a GA outperforms local search with this extra complexity.

The game has 5 reels, 20 paylines, 9 symbols and two game stages. Each stage is tuned separately. A total of 9 GA configurations are compared against hill climbing, simulated annealing and random search, over 30 paired seeds and

4 mutation strengths. Every method had 1000 evaluations, and the best reel set of every run was re-evaluated with 10^8 spins on a new seed. In the base stage, the best GA and simulated annealing reach median errors of 0.076 and 0.075, and no GA configuration is significantly better than either local search method. Generational replacement is the worst scheme: when 10 of the 250 positions are mutated, it loses to steady-state replacement on 30 of 30 seeds. In the free stage, the targets appear to conflict, and only 16 of 2310 final reel sets reach an error of 0.25 or less.

Our contributions are:

- A comparison of twelve methods for tuning reel sets, with equal budgets and paired seeds, where every finding must also be larger than the noise floor.
- Evidence that replacement decides how well a GA performs, that selection matters mainly when the best member is not kept, and that the best GA configurations only tie local search in the base stage.
- A noise study that measures how optimistic the in-run best error is, and a fixed-base check showing that the free-stage findings do not come from the base reels.

Sections 2 and 3 give the background and related work, Sects. 4–6 the problem, methods and setup, and Sects. 7–9 the results, discussion and conclusion.

2 Background: How a Video Slot Pays

A reel set has several reels, and each reel has a cyclic strip of L symbols. During a spin, the game stops each reel at a random position, independently and uniformly, and shows the symbol at the stop and the N symbols below it (Fig. 1).

In our case the reel set has 5 reels, each with a strip of length $L = 50$, i.e. $50^5 \approx 3.1 \times 10^8$ equally likely stop combinations, and we show $N = 2$ symbols below the stop, making a 5×3 window. Changing even a single symbol on a strip alters every window in which that position is visible, i.e. 6% of all windows, so a small change can cause a major shift in the stats of the game.

How a spin pays. The total bet B is divided over 20 paylines, hence the line bet is $B/20$. Three or more of the same symbol in a row on a payline, counted from the leftmost reel, pays a prize multiplied by the line bet: 20, 100 or 200 for 3, 4 or 5 of A, B or C; 5, 20 or 100 for D or E; and 1, 10 or 50 for F or G. Besides these seven regular symbols there are two special symbols. The wild (W) can substitute for any regular symbol and has its own pays (40, 400 or 1000), and the higher prize is paid when both could pay on the same line. The scatter (S) pays anywhere in the window, at 10, 50 or 500 times the total bet for 3, 4 or 5 scatters. The win of a single spin is capped at $1100B$.

Free games. Three to five scatters anywhere in the window award 10 free spins, played at no extra cost on a separate free reel set, so mathematicians have to manage two reel sets: base and free. Scatters inside the free spins award 10 additional free spins, hence a *retrigger*, so the number of free spins in a round is not fixed.

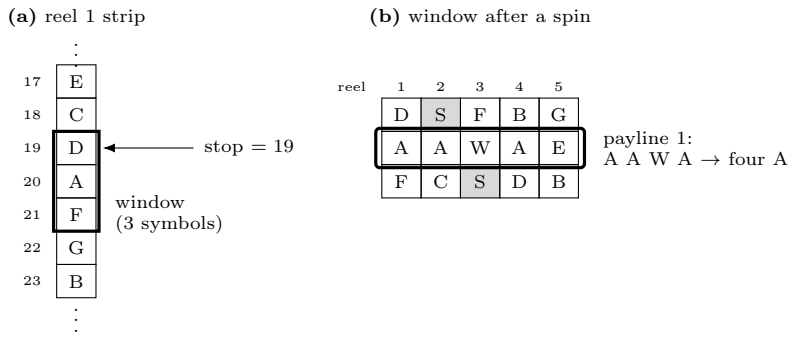


Fig. 1. (a) A spin picks a stop on each strip; the window shows that symbol and the next two. (b) The resulting window. On payline 1 the wild W completes a run of four A's, which pays 100 line bets. Only two scatters (shaded) are visible, so no free games start.

Stats a mathematician has to maintain. Return to player (RTP) is the average amount paid back to the player: a game with an RTP of 95% pays back 95 dollars for every 100 dollars staked, on average. Total RTP consists of base-game RTP and free-game RTP, both counted per base spin, and our game has a target of 0.95 (0.57 base and 0.38 free). *Hit rate* is quoted as "1 in X", i.e. on average one in every X spins wins something; for a fixed RTP, a larger X means less frequent but bigger wins. *Trigger rate* is the number of base spins per free-game trigger, and *retrigger rate* the number of free spins per retrigger, both also quoted as "1 in X". These stats are recorded in the game's PAR sheet [10]. RTP is the number regulators check, and the same game can be released with different RTPs, each with a different reel set [11].

No change touches only one stat: an extra wild can raise both RTP and hit rate, and an extra scatter makes free games come more often, adds scatter prizes and so raises the overall RTP.

3 Related Work

Game balancing and content generation. Search-based procedural content generation uses search methods, such as evolution, to create game content, and a fitness function scores each candidate [21]. Game balancing uses the same idea on an existing game. Morosan and Poli [17] balanced Ms PacMan and StarCraft with evolutionary algorithms. In our problem we have asked whether a GA, compared against local search, is the best to achieve a set of targets.

Optimisation under noise. Noisy fitness has been studied widely [12,18]. The simplest fix is resampling, i.e. evaluating a candidate several times and taking the average, but this uses up evaluations. Fitzpatrick and Grefenstette [8] found that a GA can gain more from evaluating more candidates than from averaging

more samples. Arnold and Beyer [1] showed for the (1+1) evolution strategy that a parent that is not re-evaluated gets overvalued, hence its children succeed less often, although performance can still improve. We have also measured this optimism in our project, for which we re-evaluated the best reel set of every run on a new seed with 10^8 spins. As a result, the base-stage median error came out higher than the in-run median in 36 of 44 method–mutation combinations.

Slot machine mathematics. Gambling research describes how slot games are designed: Harrigan and Dixon analysed PAR sheets, the documents that list a game's reel strips and prizes [10], and studied versions of one game with different RTPs [11]. Reel strips have also been tuned by search. Balabanov et al. [4] used a GA to reach a desired RTP by controlling the symbol distribution, Keremedchiev et al. [14] used an evolutionary algorithm on base-game RTP, and Kamanas et al. [13] used variable neighbourhood search on the base game. Even though these studies targeted RTP, we have targeted hit rate and trigger rate (retrigger rate in the free game) along with RTP on the base and free game stages.

Steady-state and generational GAs. GENITOR [22] replaces one member at a time, whereas a generational GA replaces the whole population. Syswerda [20] compared the two, and De Jong and Sarma [5] studied the share of the population replaced at each step. Goldberg and Deb [9] compared selection schemes by takeover time, i.e. the number of generations selection alone needs to fill the population with copies of the best member. Rogers and Prügel-Bennett [19] showed that genetic drift is twice as fast under steady-state selection as under generational selection. We have tested these replacement schemes under noisy evaluations, with $\mu = 10$ and four mutation strengths, and used hill climbing and simulated annealing as baselines to see when a GA's population pays off.

4 Problem Formulation

4.1 Search Space and Two Tuning Stages

A candidate solution is one reel set $x = (r_1, \ldots, r_5)$, where each strip $r_j \in \Sigma^L$ holds $L = 50$ symbols from the 9 symbols $\Sigma = \{\mathrm{A}, \ldots, \mathrm{G}, \mathrm{W}, \mathrm{S}\}$. We put no constraint on symbol counts, so a strip can hold any number of copies of a symbol, or none.

The base and free reels are tuned separately, first the base reels and then the free reels. During the base stage, the three base stats (base RTP, hit rate and trigger rate) are calculated and tuned. For the free stage the base reels are fixed and only the free reel set changes. The three free stats are free RTP, free hit rate and retrigger rate. Free RTP also depends on the base trigger rate T, but keeping the base reels constant makes it depend only on the free reels.

Table 1. Targets t_i and, in brackets, weights w_i of the objective (1). Rates are "1 in X"; the free stage has a retrigger rate.

Stage	RTP	Hit rate	(Re)trigger rate
Base	0.57 (10)	3 (6)	80 (3)
Free	0.38 (10)	2.7 (6)	60 (3)

4.2 Objective

Each stage has three stats $s_i(x)$, each with a target t_i and a weight w_i (Table 1), and we minimise the weighted sum of relative errors

$$F(x) \;=\; \sum_i w_i \,\frac{|s_i(x) - t_i|}{t_i}\,. \tag{1}$$

$F(x) = 0$ means every target stat is matched exactly. Dividing the error of each stat by its target normalises it, so that stats on very different scales, such as an RTP of 0.57 and a trigger rate of 80, can be added together. Rates enter in their "1 in X" form. RTP has the highest weight, since it is the value that testing labs and licensing bodies check, whereas the trigger rate has more room: being 15 or more spins away from the target can be acceptable. A weighted sum is the simplest way to combine several targets and is close to how a mathematician reads a PAR sheet, but it hides the trade-off when targets conflict, which happens in the free stage (Sect. 7).

4.3 Noisy Evaluation by Simulation

For this game, every stat could actually be calculated exactly: line-win RTP follows from the symbol counts on each reel, hit rate from going through all 50^5 windows, and the free game from a geometric series over the number of free spins. However, we want a tuning process that also works for cascade and cluster games, where exact maths becomes much harder. Hence, the same way mathematicians check reel sets, we treat the game as a black box and use a multithreaded C++ Monte Carlo simulator. One evaluation plays $n = 10^7$ base spins with a random seed ω and returns $\hat{F}_n(x;\omega)$, which takes about 0.15 s on 10 threads. In the free stage, the trigger rate T of the fixed base reels is measured once from 10^8 spins, and each evaluation then plays round$(10^7/T)$ free rounds directly, i.e. the number of rounds a 10^7-spin full game would play on average, and free RTP is calculated by dividing by T. This skips the base spins between free rounds and saves simulation time.

Since every stat comes from a finite number of spins, the score is noisy. Every evaluation in a run uses the same seed ω (common random numbers [16]), so the same reel set always gets the same score, and a reel set that got a lucky score keeps it. The standard deviation of one 10^7-spin estimate of F is around 0.013 for good base reel sets and 0.035 in the free stage (Sect. 7.5), which near the

Table 2. The twelve methods. μ is the population size. "Per step" is one generation or iteration. Every method spends exactly 1000 evaluations.

Method	Selection	μ	Per step	Survivors	Steps
Steady-state GA	T / R / SUS	10	$2 \to 1$	replaces the worst if better	1000
Elitist gen. GA	T / R / SUS	10	$8 \to 8$	2 best + 8 children	125
Generational GA	T / R / SUS	10	$10 \to 10$	10 children	100
Hill climbing	—	1	$1 \to 1$	child, if not worse	1000
Simulated annealing	—	1	$1 \to 1$	if better, else prob. $e^{-\Delta/\tau}$	1000
Random search	—	1	$0 \to 1$	best so far	1000

T = tournament ($k = 3$), R = rank-based roulette wheel, SUS = stochastic universal sampling. "$a \to b$": a parents selected, b children evaluated.

end of the base stage is close to the gap between good candidates. A run also reports the lowest of about 1000 noisy scores, which favours the reel set whose noise was lucky [12,18]. So for every result we report, we re-evaluated the run's best reel set with 10^8 spins on a seed the run never used (Sect. 6.2).

4.4 Initialisation and Variation Operators

A random reel set puts one copy of every symbol on each strip, fills the other 41 positions at random and shuffles the strip. Mutation picks $m \in \{1, 3, 5, 10\}$ of the 250 positions without replacement and changes each one to a different random symbol. The GA, hill climbing and simulated annealing all use this same mutation, so any difference between them comes from the population, crossover and the acceptance rule. Crossover works reel by reel: for each reel a cut point $c \in \{1, \ldots, L-1\}$ is drawn, and the child takes the first c symbols from one parent and the rest from the other. Every child gets crossover first and then mutation.

5 Methods

We compare twelve methods: nine GA configurations (three replacement schemes $\times$ three selection operators) and three baselines, i.e. random search, hill climbing and simulated annealing (Table 2). All twelve use the same representation, initialisation and mutation (Sect. 4.4), the same objective (Sect. 4.2) and the same budget of 1000 evaluations.

5.1 The Genetic Algorithm

The GA starts with 10 random reel sets, the same 10 for every GA combination and for the baselines, and each of them is evaluated once. In each generation, parents are selected, children are created by crossover, each child is mutated and evaluated, and then it is decided which members of the population are replaced.

The GA keeps track of the best reel set it has seen so far, since a scheme such as generational replacement can lose its best member.

Selection. All three selection methods only consider the ranking of the population, not the size of the errors, because F is around 90 or more at the beginning and falls below 0.1 at the end, so fitness-proportional selection would work very differently at the start and at the end of a run. *Tournament* ($k = 3$) draws three distinct members at random and selects the best; each parent comes from its own tournament. *Roulette* uses Baker's linear ranking [2] with selection pressure $s = 2$, so the best member gets a weight of 2 and the worst a weight of 0, and chooses a single parent per spin. *SUS* [3] uses the same weights, but places n pointers μ/n apart from a single random start, where n is the number of parents. So even though Roulette and SUS have the same expected selection count, Roulette can by chance leave the best member out entirely, whereas SUS always selects it when 8 or 10 parents are drawn.

Replacement. The three schemes range from replacing a single worst member to replacing all the members [5]. *Steady-state* selects 2 parents and produces a single child, which replaces the worst member if it is strictly better, hence the best member is never lost [22,20]. *Generational* selects 10 parents and pairs them, each pair creates two children, and the 10 children replace the whole population, hence the best genes can be lost. *Elitist generational* copies the 2 best members as they are and replaces the other 8 by 8 children of 8 parents selected from the whole population.

5.2 Baselines

Random search draws a new random reel set at every evaluation and keeps the best. It uses no mutation, so it is run once per seed and reported under every mutation strength.

Hill climbing is the (1+1) EA [6]. It takes the best reel set among the initial 10, mutates it, and replaces it with the child if the child is at least as good. Apart from accepting ties, hill climbing is the same as a steady-state GA with a population of 1 and no crossover. That makes it the main baseline the GA has to beat: since both use the same mutation, beating hill climbing shows that the population and crossover give an advantage in tuning.

Simulated annealing [15] is the same as hill climbing, but it can also accept a worse child y of the current reel set x, with probability $\exp(-\Delta/\tau)$, where $\Delta = \ln \hat{F}(y) - \ln \hat{F}(x)$. The log scale is used because the score falls by about three orders of magnitude during a run, so on a linear scale a fixed temperature would either accept far too much at the end or almost nothing at the beginning. The temperature falls geometrically from $\tau_0 = 0.14$ at the first evaluation to $\tau_{\text{end}} = 0.002$ at the last, and the method returns the best reel set it has seen.

5.3 Equal Budgets and Parameter Choices

Same budget. Every method spends exactly 1000 evaluations after the initial population (Table 2). In the base stage and the fixed-base check, the ten evaluations of the initial population are made once per seed and shared; in the main free stage each method re-evaluates them on its own base reels. The GA starts from all ten and the baselines from the best of them. In the base stage every evaluation plays exactly 10^7 spins. In the free stage each evaluation plays round$(10^7/T)$ free rounds, which differs between methods, since each method's free stage uses the base reels that method found: a median of about 125,500 rounds for the GA and local-search methods, but about 560,000 for random search, whose base reels trigger free games about every 18 spins instead of every 80. The fixed-base check of Sect. 6.2 removes this difference.

Parameters. No method was tuned. The GA settings ($\mu = 10$, $k = 3$, $s = 2$, two elites, crossover on every child) are common textbook defaults [7]. The annealing temperatures were set once, when the baselines were added to the code, and were not changed afterwards: at $\tau_0 = 0.14$ a child 10% worse is accepted with probability $\exp(-\ln 1.1/0.14) \approx 0.51$, and at $\tau_{\text{end}} = 0.002$ even a child 1% worse is accepted with probability below 0.01, so the search ends as a hill climber. So the comparison is between sensible default settings, not between methods each tuned to its limit (Sect. 8).

6 Experimental Setup

6.1 Design

Every method was run on 30 seeds at each of the four mutation strengths $m \in \{1, 3, 5, 10\}$. Random search does not use mutation, hence it is run once per seed and stage, which gives 1,350 runs per stage. Each seed runs the base stage first and then the free stage, where the free stage of each method uses the base reels that method found. During the base stage free games are triggered and counted for the trigger rate, but not played, since the free reels are still random at that point. For each seed, every method starts from the same 10 random reel sets and uses the same simulator seed for every evaluation, the same mutation operator and strength, and the same budget, so on any seed, all twelve methods face exactly the same problem, and all the tests below compare methods seed by seed. Every random number in a run is derived from the seed number, so any run can be repeated exactly.

Computing. A run of 1000 evaluations plus its re-evaluation takes about 2.5 minutes on 10 threads, and the main study took 83 run-hours, about 14 hours on a 64-core machine running 6 seeds in parallel. The code, the results and the simulator are available at `https://osf.io/bw32t/overview?view_only=06e35aa867254e2da2c94bcfc0450f67`.

6.2 Re-evaluation and Fixed-Base Check

As explained in Sect. 4.3, the best score seen inside a run is optimistic. Hence the best reel set of every run is re-simulated with 10^8 spins, i.e. ten times the spins of one evaluation, on a seed the run never used. In the base stage this re-evaluation plays the base game only, and in the free stage it plays the full game. Every error we report in Sect. 7 is this re-evaluated value.

In the free stage, each method was run using the best base reel set found during its own base-stage run, which can make the free-stage comparison unfair, as some methods start with worse base reels than others. To make the comparison fair, all methods were later run again using the same fixed base reel set, the best one found over all base-stage runs. This fixed-base check covers $m \in \{3, 5, 10\}$ and all eleven methods other than random search, i.e. $30 \times 3 \times 11 = 990$ runs. Random search was left out because it is already far behind every other method in the free stage (median error 62.8, against about 1.6 at $m = 3$) and its fully random free reels make every evaluation slow.

6.3 Statistical Tests

Methods are compared in families, separately for each mutation strength: the three replacement schemes (per seed, the median over their three selection methods), the three selection methods inside each replacement scheme, the nine GA configurations against the three baselines, and the mutation strengths inside each method. Pairs are compared with two-sided Wilcoxon signed-rank tests paired by seed, with Holm correction inside the family and $\alpha = 0.05$. We also report a Friedman test per family, but since Holm correction already controls the chance of any false positive in the family, the pairwise results are not made conditional on it. As effect size we report the Vargha–Delaney A_{12}, i.e. the probability that one method ends with a lower error than the other on a run, where 0.5 means no difference, with the usual bands: negligible below 0.56, small from 0.56, medium from 0.64 and large from 0.71.

Some differences can be significant and still smaller than what the simulator can measure. Two independent measurements have to differ by $1.96\sqrt{2} \approx 2.77$ standard deviations to be told apart at 95%. For the noise floor we use a standard deviation of 0.034 for the base stage and 0.10 for the free stage at 10^7 spins, fixed from an earlier noise measurement before the main experiment. Both are above the largest values in the final noise study (0.030 and 0.093 for random search's reel sets, 0.025 and 0.061 for the other methods), so the floor is conservative. This gives floors of $2.77 \times 0.034/\sqrt{10} = 0.0298$ and $2.77 \times 0.10/\sqrt{10} = 0.088$ at the 10^8 spins of the re-evaluation. A difference counts as a finding only if it is significant after Holm correction, its A_{12} is at least small, and the gap between the medians is above the noise floor.

7 Results

All errors in this section are re-evaluated errors (Sect. 6.2), given as medians over the 30 seeds, and lower is better. Table 3 gives every method at every mutation strength for both stages.

Table 3. Median re-evaluated error over 30 seeds (lower is better). SS = steady-state, EL = elitist generational, GEN = generational. Bold: within one noise floor of the best value in its column (0.0298 base, 0.088 free, Sect. 6.3).

	Base stage, $m =$				Free stage, $m =$			
Method	1	3	5	10	1	3	5	10
SS + Tournament	**0.076**	**0.114**	**0.090**	**0.139**	2.032	**1.303**	1.347	1.797
SS + Roulette	0.163	0.137	0.150	0.206	2.008	1.356	1.718	1.766
SS + SUS	0.140	0.153	**0.091**	0.169	2.072	**1.262**	1.489	1.593
EL + Tournament	0.176	0.119	**0.100**	0.210	2.021	1.423	**1.174**	1.601
EL + Roulette	0.155	**0.086**	0.132	0.269	1.940	1.363	1.474	**1.574**
EL + SUS	0.135	0.151	0.148	0.221	**1.828**	1.479	1.431	1.730
GEN + Tournament	0.169	0.158	0.209	0.522	2.010	1.706	1.870	1.990
GEN + Roulette	0.252	0.349	0.445	1.555	2.069	1.907	2.001	2.059
GEN + SUS	0.173	0.234	0.333	0.830	**1.885**	1.729	1.900	1.952
Hill climbing	0.160	0.118	**0.084**	**0.128**	1.988	1.780	1.559	**1.527**
Simulated annealing	0.192	0.130	**0.075**	**0.155**	1.926	1.532	1.277	**1.494**
Random search	37.5	37.5	37.5	37.5	62.8	62.8	62.8	62.8

7.1 Automatic Tuning Works

In the base stage, the best of the 10 random starting reel sets has a median error of about 97. With 1000 evaluations, every method except random search brings this down to between 0.075 and 0.25 at its best mutation strength, and most of them to around 0.1 (Table 3). An error of 0.1 means, for example, a base RTP that is 1% off its target with the other two stats exact. Random search, with the same budget, stays at 37.5 and loses to every GA configuration on 30 of 30 seeds at every mutation strength ($p_{\text{Holm}} < 0.001$, $A_{12} = 1.00$), so the result comes from the search itself and not from luck.

7.2 Replacement, Selection and Mutation

Replacement decides the result. Generational replacement is the worst scheme, and the gap grows with mutation strength. Against steady-state, its median error is 0.185 vs 0.145 at $m = 1$, 0.214 vs 0.144 at $m = 3$, 0.333 vs 0.144 at $m = 5$ and 0.707 vs 0.162 at $m = 10$, where it loses on 30 of 30 seeds ($A_{12} = 1.00$, $p_{\text{Holm}} < 0.001$). Elitist replacement beats generational in the same

way from $m = 3$ upwards. The result at $m = 1$ is the weakest one ($p_{\text{Holm}} = 0.044$, Friedman $p = 0.107$). Steady-state and elitist replacement are tied at $m = 1$, 3 and 5. Only at $m = 10$ is steady-state better (0.162 vs 0.224, $p_{\text{Holm}} = 0.007$, $A_{12} = 0.72$).

Selection matters only without elitism. Inside steady-state and inside elitist replacement, no pair of selection methods differs significantly at any mutation strength. Inside generational replacement, selection does matter: Tournament beats Roulette at every m, SUS beats Roulette at every m, and Tournament beats SUS from $m = 3$ upwards.

Gentle mutation is better. Changing 10 of the 250 positions is significantly worse than a smaller m for 9 of the 11 methods. The damage is largest for generational replacement: Generational + Roulette goes from 0.252 at $m = 1$ to 1.555 at $m = 10$.

7.3 GA versus Local Search

In the base stage, no GA configuration is significantly better than hill climbing or simulated annealing at any mutation strength. With each method at its own best mutation strength, the best GA (SS + Tournament, $m = 1$) and simulated annealing ($m = 5$) reach 0.076 and 0.075, simulated annealing is lower on 15 of the 30 seeds, $A_{12} = 0.53$ and $p_{\text{Holm}} = 1.0$, i.e. a draw. In the other direction, the baselines are significantly better than every generational configuration at $m = 5$ and 10, so in the base stage the best a well-configured GA can do is a draw, and a badly configured GA loses to local search.

7.4 Free Game and Fixed-Base Check

The free stage has a higher error, with best medians of 1.17 to 1.30 against about 0.1 in the base stage, because the free-game targets pull against each other (Fig. 2). The ranking of methods is similar: steady-state and elitist replacement are at the top, level with simulated annealing, and generational replacement is significantly worse than both at $m = 3$, 5 and 10 (A_{12} from 0.80 to 0.94). Three things differ. First, $m = 1$ is now the worst mutation strength for 10 of the 11 methods. Second, elitist replacement beats steady-state at $m = 1$ and 5 ($p_{\text{Holm}} = 0.030$ and 0.009, $A_{12} = 0.65$). Third, all six steady-state and elitist configurations beat hill climbing at $m = 3$, e.g. SS + SUS with 1.262 vs 1.780 on 27 of 30 seeds ($A_{12} = 0.83$, $p_{\text{Holm}} < 0.001$), whereas no GA configuration beats simulated annealing at any m. Selection again matters mainly inside generational replacement (one exception: EL + SUS beat EL + Tournament at $m = 1$, $p_{\text{Holm}} = 0.034$).

With the fixed base reels, the median error over the 990 paired runs barely moves (1.661 vs 1.649, Wilcoxon $p = 0.77$), the ranking is kept (Spearman $\rho = 0.84$), and the same findings hold, except that only three configurations

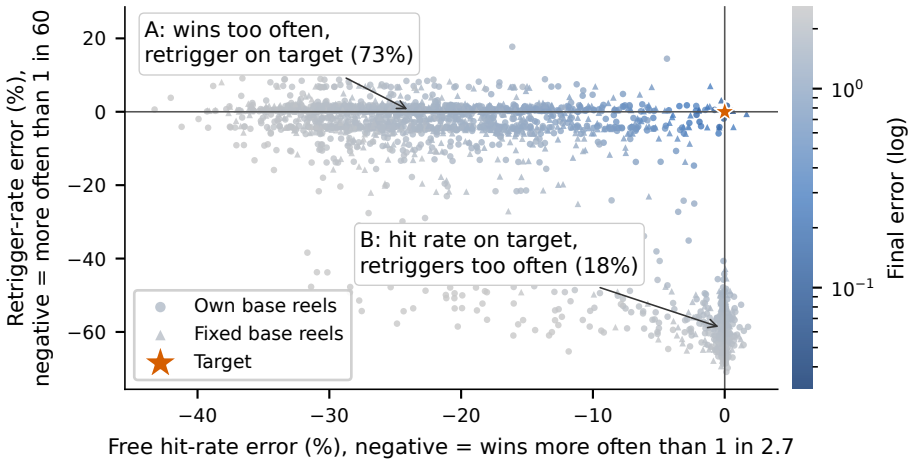


Fig. 2. Free stage: error in free hit rate against error in retrigger rate (in %) for the 2310 final free reel sets, with each method's own base reels (circles) and with the fixed base reels (triangles), random search excluded. Colour shows the final error; the target is the star at (0, 0). Most reel sets fall into two groups: A hits the retrigger target but wins too often, and B hits the hit-rate target but retriggers too often.

beat hill climbing at $m = 3$ (SS + SUS, SS + Tournament and EL + SUS). So the free-stage findings do not come from the base reels.

Of the 2310 final free reel sets, 73% hit the retrigger target but win too often, 18% hit the free hit-rate target but retrigger far too often, and only 16 (0.7%) reach an error of 0.25 or less. Every retrigger adds 10 spins, so more retriggers mean longer rounds and more free RTP. With this game's symbols and rules the free-game targets appear to conflict, and every method has to trade one against the other.

7.5 Noise Study

Re-simulating the best reel sets 30 times each with 10^7 spins, the standard deviation of the error is about 0.013 in the base stage and 0.035 in the free stage (medians over reel sets), almost the same for every method except random search, since it comes from the simulator and not from the optimiser. For a given method and m, the spread of final errors between seeds (interquartile range) is a median 4.3 times the noise floor in the base stage and 5.6 times in the free stage, so the outcome of a run is decided by the search, not by measurement error. The in-run best is optimistic: in the base stage the re-evaluated median is higher than the in-run median in 36 of 44 method–mutation combinations (median ratio 1.04), and in one case by a factor of 1.63 (SS + Tournament at $m = 1$: 0.047 during the run, 0.076 on re-evaluation); in the free stage the ratios lie between 0.98 and 1.07. Finally, every significant difference reported above is also larger than the noise floor: all 75 in the base stage, 67 in the free stage and

27 in the fixed-base check. However, noise does limit the order among the best GA configurations and the baselines, as their differences are within the noise floor in the base stage, and that is why they are reported as tied rather than ranked.

8 Discussion and Threats to Validity

Why generational replacement fails. Generational replacement replaces all the parents, meaning no reel set survives unchanged. Near the end of a run, when the reel sets are close to the targets, the children are mostly worse than their parents, and a higher mutation strength has a more severe impact. Unlike generational replacement, steady-state and elitist replacement never lose the best reel set, so a bad child costs just one evaluation, nothing more.

Selection only matters when replacement does not keep the best. With steady-state and elitist replacement the best reel sets are always kept, but with generational replacement, selection is the only way good reel strips reach the next generation. When 10 parents are drawn from 10 members, Tournament ($k = 3$) picks the best member 3 times on average and misses it in $0.7^{10} \approx 3\%$ of generations. SUS picks it exactly twice. Roulette also picks it twice on average, but misses it in $0.8^{10} \approx 11\%$ of generations.

In the base stage, local search is enough. Hill climbing is a steady-state GA with one member and no crossover (Sect. 5.2), so the only way the GA can beat it is if crossover or the population helps, but in the base stage neither did. Line-win RTP depends only on how many of each symbol a reel strip holds (Sect. 4.3), and one mutation changes two of these counts by 1 in 50. So even without crossover, a chain of small mutation steps lowers the error. We think crossover adds very little near the end, but we never measured diversity or ran a GA without crossover.

In the free stage, a population and annealing help. The free-game target stats conflict with each other (Sect. 7.4), so near the end every small change likely makes at least one of the stats worse. Since hill climbing only accepts a child that is not worse, it stays there. At $m = 3$ all six steady-state and elitist configurations beat it. Simulated annealing also works on a single reel set, but it accepts some worse children, and no GA configuration beats it at any m. So the gain seems to come from keeping or accepting reel sets other than the current best, which a population and annealing both do, and not from crossover.

Advice for slot mathematicians. Start with a steady-state GA, an elitist GA or simulated annealing; all three reach a base error of about 0.1, e.g. a base RTP 1% off its target. Do not use generational replacement without elitism. Changing 3 to 5 of the 250 symbols is a safe mutation strength for both stages. Also, re-simulate the final reel set with more spins on a new seed. Finally, if every good method stops at about the same error, as in our free stage (best medians 1.17 to 1.30), check the targets before changing the optimiser.

Threats to validity. One game, one budget, one population size. We tuned a simple line-win game against three stats per stage. More complex games may favour other methods. Every run had 1000 evaluations and $\mu = 10$. A larger budget might let crossover pay off, and a larger population gives more diversity but fewer generations; we tested neither. *Untuned methods.* The GA used textbook defaults, and the annealing temperatures (0.14 to 0.002) were set once and never changed. Tuning could change the order among the tied methods, so our claim is about sensible defaults. *Best m chosen on the same seeds.* When each method is compared at its own best m, the same 30 seeds choose m and measure the result. This favours the GA, whose best value is the lowest of 36 configurations (9×4 values of m) against 4 for simulated annealing. The base-stage draw holds despite this, and at a fixed $m = 5$ it is also a draw. *Weighted-sum error and conflicting free targets.* The weights 10, 6 and 3 are our choice, and other weights could rank reel sets and methods differently. Only 16 of 2310 final free reel sets reach an error of 0.25 or less, so the free stage compares how methods trade one target against another. We did not prove that the free targets cannot all be met; the exact calculation of Sect. 4.3 could settle this. *Noise floor.* Its standard deviations (0.034 and 0.10) come from a measurement made before the main experiment, and are above every value in the final noise study.

9 Conclusion

We have compared 9 configurations of GA with hill climbing, simulated annealing and random search. Every method was given an equal budget of 1000 evaluations, and all of them were run on 30 seeds and 4 mutation strengths. In the base stage, no GA configuration could conclusively beat local search, with the best GA coming as close as a median error of 0.076 against simulated annealing's 0.075. Generational replacement was worse, losing to steady-state replacement on 30 of 30 seeds at $m = 10$. In the base stage, every generational configuration lost to local search at $m = 5$ and 10. In the free stage, all six steady-state and elitist configurations beat hill climbing at $m = 3$, but no GA could beat simulated annealing at any m.

Four steps to follow. Multi-objective tuning of the free game could show the trade-off between its targets, since only 16 of 2310 final reel sets reached an error of 0.25 or less. More complex bonus features such as hold-and-spin and game types such as cascade and cluster would test whether these results hold elsewhere. Tuned baselines would show whether the order among the tied methods changes. A GA without crossover, together with a diversity measure, would show whether the population or crossover produces the free-stage gain.

Use of AI Tools. AI tools helped with the code, the analysis scripts and the writing.

References

1. Arnold, D.V., Beyer, H.G.: Local performance of the (1+1)-ES in a noisy environment. IEEE Transactions on Evolutionary Computation **6**(1), 30–41 (2002). https://doi.org/10.1109/4235.985690
2. Baker, J.E.: Adaptive selection methods for genetic algorithms. In: Proceedings of the First International Conference on Genetic Algorithms and Their Applications. pp. 101–111. Lawrence Erlbaum (1985)
3. Baker, J.E.: Reducing bias and inefficiency in the selection algorithm. In: Proceedings of the Second International Conference on Genetic Algorithms. pp. 14–21. Lawrence Erlbaum (1987)
4. Balabanov, T., Zankinski, I., Shumanov, B.: Slot machines RTP optimization with genetic algorithms. In: Numerical Methods and Applications. Lecture Notes in Computer Science, vol. 8962, pp. 55–61. Springer (2014). https://doi.org/10.1007/978-3-319-15585-2_6
5. De Jong, K.A., Sarma, J.: Generation gaps revisited. In: Foundations of Genetic Algorithms 2. pp. 19–28. Morgan Kaufmann (1992)
6. Droste, S., Jansen, T., Wegener, I.: On the analysis of the (1+1) evolutionary algorithm. Theoretical Computer Science **276**(1–2), 51–81 (2002)
7. Eiben, A.E., Smith, J.E.: Introduction to Evolutionary Computing. Natural Computing Series, Springer, 2nd edn. (2015)
8. Fitzpatrick, J.M., Grefenstette, J.J.: Genetic algorithms in noisy environments. Machine Learning **3**(2–3), 101–120 (1988). https://doi.org/10.1007/BF00113893
9. Goldberg, D.E., Deb, K.: A comparative analysis of selection schemes used in genetic algorithms. In: Foundations of Genetic Algorithms. pp. 69–93. Morgan Kaufmann (1990). https://doi.org/10.1016/B978-0-08-050684-5.50008-2
10. Harrigan, K.A., Dixon, M.: PAR sheets, probabilities, and slot machine play: Implications for problem and non-problem gambling. Journal of Gambling Issues **23**, 81–110 (2009). https://doi.org/10.4309/jgi.2009.23.5
11. Harrigan, K.A., Dixon, M.: Government sanctioned "tight" and "loose" slot machines: How having multiple versions of the same slot machine game may impact problem gambling. Journal of Gambling Studies **26**(1), 159–174 (2010)
12. Jin, Y., Branke, J.: Evolutionary optimization in uncertain environments – a survey. IEEE Transactions on Evolutionary Computation **9**(3), 303–317 (2005)
13. Kamanas, P.A., Sifaleras, A., Samaras, N.: Slot machine RTP optimization using variable neighborhood search. Mathematical Problems in Engineering **2021**, 1–8 (2021). https://doi.org/10.1155/2021/8784065
14. Keremedchiev, D., Tomov, P., Barova, M.: Slot machine base game evolutionary RTP optimization. In: Numerical Analysis and Its Applications. Lecture Notes in Computer Science, vol. 10187, pp. 406–413. Springer (2016). https://doi.org/10.1007/978-3-319-57099-0_45
15. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983)
16. Law, A.M.: Simulation Modeling and Analysis. McGraw-Hill, 5th edn. (2015)
17. Morosan, M., Poli, R.: Automated game balancing in Ms PacMan and StarCraft using evolutionary algorithms. In: Applications of Evolutionary Computation. Lecture Notes in Computer Science, vol. 10199, pp. 377–392. Springer (2017). https://doi.org/10.1007/978-3-319-55849-3_25
18. Rakshit, P., Konar, A., Das, S.: Noisy evolutionary optimization algorithms – a comprehensive survey. Swarm and Evolutionary Computation **33**, 18–45 (2017)

19. Rogers, A., Prügel-Bennett, A.: Genetic drift in genetic algorithm selection schemes. IEEE Transactions on Evolutionary Computation **3**(4), 298–303 (1999). https://doi.org/10.1109/4235.797972
20. Syswerda, G.: A study of reproduction in generational and steady-state genetic algorithms. In: Foundations of Genetic Algorithms. pp. 94–101. Morgan Kaufmann (1990)
21. Togelius, J., Yannakakis, G.N., Stanley, K.O., Browne, C.: Search-based procedural content generation: A taxonomy and survey. IEEE Transactions on Computational Intelligence and AI in Games **3**(3), 172–186 (2011). https://doi.org/10.1109/TCIAIG.2011.2148116
22. Whitley, D.: The GENITOR algorithm and selection pressure: Why rank-based allocation of reproductive trials is best. In: Proceedings of the Third International Conference on Genetic Algorithms. pp. 116–121. Morgan Kaufmann (1989)